Nabbit — Product Requirements Document

Train your code-review judgment on the dev videos you watch.

Oct 20, 2026 · @Sagnik · HelloPM Buildathon entry (Cohort 52)

At a glance

Field	Value
Product	**Nabbit** — "catch the nit"
One line	Turn a dev tutorial into a bug-hunt drill that trains your code-review judgment. (13 words)
Primary user	A working or self-learning engineer who increasingly reviews AI-written code
Domain	Developer learning / upskilling
Status	MVP build · HelloPM Buildathon entry (Cohort 52)
Build platform	React + Express (reusing the Recap base) · Gemini for content · deployed on Vercel
MVP scope (P0)	Pick a curated dev video → one bug-hunt challenge on its key concept → spot & fix it → it resurfaces for spaced recall
Not in MVP	Freeform code editor · auto-generation from arbitrary videos · other challenge formats · accounts & payments

Problem & User

Engineers watch dev tutorials, feel they understood them, then can't catch problems in that same code weeks later — and nothing brings the concept back to practise it.

Persona. "Arin", a backend or full-stack engineer with 2–6 years' experience, picking up a new framework or library from YouTube tutorials. Watches 2–4 tutorials a week, usually around a task or a new tool. Increasingly *reviews* AI-written code (PRs, Copilot / Claude output) rather than writing every line by hand. Tools today: YouTube, their IDE, the tutorial's GitHub repo, and rewatching. At work, judged on shipping correct code and catching issues in review.

Evidence. My own learning workflow plus conversations with engineers on my team; to confirm with 3–5 interviews before building deeper (assumption).

Today's workflow

#	Step	Pain
1	Watches a tutorial, follows along	Passive — feels understood in the moment
2	Later, reviews a PR or AI-generated code using that concept	Can't tell what's subtly wrong
3	Rewatches, Googles, or re-clones the tutorial repo	Slow, and it still doesn't stick
4	Forgets the concept within days	No resurfacing, no practice loop

Why now. AI writes the first draft of code, so the scarce skill is shifting from *writing* syntax to *reading, reviewing and verifying* it — exactly the muscle passive tutorials never train.

Why this approach, not the alternatives

The wedge in one line: **nobody lets you practise *reviewing* code tied to the specific video you just watched, with retention built in.**

Alternative	What it does	Why it leaves the gap
ChatGPT / NotebookLM	Summarise the video, answer questions	Passive — no practice, and no code you have to *judge*; NotebookLM is also weak on technical content
LeetCode	Algorithm drills	Great practice, but disconnected from what you're learning, and it trains *writing from scratch* — the declining skill
Scrimba / Codecademy / Boot.dev	Interactive learn-by-doing	Only *their* curated content, never the video you chose
"Just rebuild the tutorial yourself"	Re-type the project in your editor	High friction; most people don't, and there's no retention loop

Nabbit drills the *growing* skill — reviewing and judging code — on the exact concept from the video, and resurfaces it until it sticks.

Scope & Use Cases

Priority	User story	Why
P0	Pick a curated dev video and get one bug-hunt challenge on its key concept	Core value
P0	Spot / fix the planted flaw and get instant pass/fail	The practice loop
P0	See an explanation of *why* it was wrong, tied to the concept	Turns a guess into learning
P0	A solved challenge resurfaces for recall after a few days	Retention is the whole point
P1	See my streak and which concepts are "at risk"	Engagement; brings me back
P1	Browse a small library of curated videos by stack	Choice without choice-paralysis
P2	Paste my own video URL	The auto-generation path (stretch)
P2	Earn XP / levels as I clear challenges	Game feel

Not in MVP (say it out loud in the demo):

- Freeform code editor + sandbox / test-runner
- Auto-generating challenges from arbitrary videos
- Other challenge formats (PR-review, drag-drop flow, predict-output)
- Accounts, teams, leaderboards, payments
- Native mobile app

The core loop & architecture

The one journey (first-time user, start to "aha" in under two minutes):

1. Pick a curated dev video from the library.
2. See its key concept and the timestamp it's taught at.

3. Get **one bug-hunt challenge**: AI-written code for that concept with a single planted flaw.
4. Spot the bad line and pick the correct fix (multiple choice).
5. Get instant pass/fail plus an explanation of *why*, tied to the concept.
6. The solved challenge is saved and scheduled to resurface (1 / 3 / 7 days).

Architecture (reusing the Recap base):

- **Frontend** (React + Vite): video + concept panel, the challenge UI (code block, options, reveal), and the recall dashboard.
- **Backend** (Express): serves curated challenges, records attempts and the recall schedule.
- **Content store**: the curated pack as structured JSON (video → concept → buggy code → options → answer → explanation).
- **Persistence**: a hosted store (Vercel KV) so progress survives serverless restarts.
- **Grading**: deterministic — compare the chosen option to the correct index. No sandbox, no LLM at request time.
- **Spaced recall**: the Recap resurfacing engine, reused as-is.

Challenge format & content

Format: bug hunt. Each challenge shows realistic, AI-plausible code for one concept with a single planted flaw. The user picks the buggy line and the correct fix from four options. Grading is a one-line equality check against the correct index — **deterministic, no code sandbox, no flaky LLM test-runner.** That removes the biggest build risk and demos cold in two minutes.

Content shape (one entry per challenge):

```
{ "videoId", "concept", "chapter", "buggyCode", "buggyLines", "fixOptions",
"correctOption", "explanation" }
```

Curated pack — demo set. Start with *React Tutorial for Beginners* (Programming with Mosh): four challenges, each mapped to a chapter.

1. **State batching** — two `setCount(count + 1)` calls add 1, not 2 (stale snapshot).
2. **Missing list key** — `items.map` with no `key`; the trap option is `key={index}`.
3. **The `&&` zero trap** — `{items.length && …}` renders a literal 0 when empty.
4. **Handler invoked on render** — `onClick={onShow()}` fires on every render.

Two more curated videos (a JS async / Promises tutorial and a `useEffect` deep-dive) complete the demo set, authored the same way. Curated-first means you control concept-to-challenge quality and demo safely, without live transcript extraction.

Metrics & Evaluation

North-star: revisit rate — the share of solved challenges a user comes back and re-solves. It's the proof the retention loop works, and it's already tracked in the Recap base.

Metric	What it tells us	Target (MVP)
Activation	User solves their first challenge from a video	< 2 min, > 70% of first sessions
Revisit rate (north-star)	The retention loop is working	≥ 1 in 3 challenges re-solved
First-try completion	Challenges are the right difficulty	60–80% solved first try

Content quality check (before any challenge ships): exactly one correct option; the flaw is real and tied to the concept; the explanation names *why*; a second person solves it correctly.

Failure modes & honest limits

Saying these in the demo reads as confidence, not weakness.

Risk / limit	How it's handled
Curated-only, not any video yet	Deliberate for the MVP — a hand-picked set keeps challenge quality high and the demo safe
Live transcript extraction is unreliable	Dev screencasts hide code on-screen and YouTube blocks server-side pulls, so the MVP never depends on it; content is pre-authored
A challenge has a wrong or ambiguous answer	Quality check before shipping: exactly one correct option, verified by a second solve
Grading edge cases	Deterministic MCQ sidesteps them for the MVP; freeform-code grading is deferred
Empty / cold state	First run shows a ready video and one challenge — value before any setup or login

Risk / limit	How it's handled
Gemini cost / limits	Challenges are pre-generated and cached, never generated at request time

Build plan & stack

Stack. React + Vite + Express (the Recap base) · deterministic MCQ grading (no sandbox) · Vercel KV for persistence · Gemini used offline to author content · deployed on Vercel.

1.5-day build sequence:

1. New repo + rebrand to Nabbit, and **deploy a hello-world to Vercel in hour one** (kills end-of-week deploy panic).
2. Swap the file-based store for Vercel KV — the one real blocker on serverless.
3. Strip the PM content; wire the **bug-hunt loop** end-to-end on one curated video.
4. Add the two remaining curated videos and the spaced-recall resurfacing.
5. Polish the single journey; demo-test on yourself, then 3–5 engineers.

Deploy-on-day-one is the rule: an early live link means no scramble at submission.

Go-to-market & AARRR

First 10 users: 3–5 engineers on your team plus 2–3 dev friends learning something new, then a post in r/learnprogramming and a dev Discord. Name the people before launch.

Stage	Answer
Acquisition	Your team + dev friends → r/learnprogramming, r/webdev, dev Discords, a Show HN
Activation	First challenge solved from a video in under two minutes
Retention	Spaced resurfacing of "at-risk" concepts, plus a streak
Referral	Share a challenge ("can you catch this bug?") or your solved streak
Revenue	Free on curated content; paid for paste-any-video auto-gen, more stacks, and team dashboards

Path to scale: 1,000 via dev communities and "bug of the day" content; 100k is a direction carried by the paste-any-video feature — not promised for the MVP.

Roadmap

The MVP is one judgment format on curated videos. What's next, in order:

1. **PR review** — approve or request changes on the AI's whole implementation (richer than a single bug).
2. **More judgment formats** — predict-the-output, spot-the-security-flaw, tradeoff-call, drag-drop the architecture; match the format to the video type.
3. **Auto-generation from arbitrary videos** — the paste-any-URL path, needing a transcript source and vision for on-screen code.
4. **More stacks and languages** — beyond React / JS to Python, backend, and systems-design videos.
5. **Mastery map + XP** — deeper gamification and a per-concept mastery view.